Proyecto: Alke Wallet

Evaluación Módulo – Fundamentos de Bases de Datos Relacionales

1. Introducción

En la actualidad, las billeteras digitales o *wallets* permiten a los usuarios gestionar su dinero de forma digital, realizar transferencias y consultar sus movimientos financieros. Estas plataformas requieren sistemas confiables que garanticen la seguridad y la integridad de los datos almacenados.

El presente proyecto tiene como objetivo diseñar e implementar una base de datos relacional para el sistema **Alke Wallet**, la cual permitirá gestionar usuarios, monedas y transacciones dentro de la plataforma.

Para desarrollar este sistema se utilizó **SQL** mediante **MySQL Workbench**, permitiendo crear las estructuras necesarias para almacenar información y ejecutar consultas para manipular los datos. Durante el desarrollo se aplicaron conceptos fundamentales de bases de datos relacionales como claves primarias, claves foráneas, integridad referencial y sentencias DDL y DML.

2. Modelo Entidad-Relación

Para el diseño del sistema se identificaron tres entidades principales:

Usuario
Representa a cada persona que utiliza la plataforma Alke Wallet. Cada usuario posee información personal y un saldo disponible dentro de la aplicación.

Moneda
Representa los distintos tipos de monedas que pueden utilizarse dentro del sistema para realizar transacciones.

Transacción
Representa cada operación financiera realizada entre usuarios dentro del sistema.

Relaciones entre entidades

Las relaciones del sistema son las siguientes:

- Un **usuario** puede realizar muchas **transacciones**.
- Un **usuario** puede recibir muchas **transacciones**.
- Cada **transacción** utiliza una **moneda** específica.
- Una **moneda** puede estar asociada a múltiples **transacciones**.

Este modelo permite mantener la integridad de los datos y garantizar una correcta organización de la información.

(Aquí se debe insertar la imagen del diagrama Entidad-Relación).

3. Creación de la Base de Datos

El primer paso fue crear la base de datos llamada **AlkeWallet**, la cual almacenará toda la información del sistema.

```
CREATE DATABASE AlkeWallet;
USE AlkeWallet;
```

Este comando crea la base de datos y posteriormente la selecciona para comenzar a trabajar en ella.

4. Definición de Tablas (DDL)

A continuación se crearon las tablas necesarias para almacenar la información del sistema.

Tabla Usuario

Esta tabla almacena la información de cada usuario registrado en la plataforma.

```
CREATE TABLE usuario (
   user_id INT AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL,
   correo VARCHAR(100) NOT NULL UNIQUE,
   contrasena VARCHAR(100) NOT NULL,
   saldo DECIMAL(10,2) NOT NULL
);
```

Explicación de los campos

Campo	Descripción
user_id	Identificador único del usuario
nombre	Nombre del usuario
correo	Correo electrónico
password	Contraseña de acceso
saldo	Dinero disponible en la wallet

Tabla Moneda

Esta tabla almacena las monedas disponibles dentro del sistema.

```
CREATE TABLE moneda (
   currency_id INT AUTO_INCREMENT PRIMARY KEY,
   currency_name VARCHAR(50) NOT NULL,
   currency_symbol VARCHAR(10) NOT NULL
);
```

Campos

Campo	Descripción
currency_id	Identificador de la moneda
currency_name	Nombre de la moneda
currency_symbol	Símbolo de la moneda

Tabla Transacción

Esta tabla almacena cada operación financiera realizada dentro de la plataforma.

```
CREATE TABLE transaccion (
   transaction_id INT AUTO_INCREMENT PRIMARY KEY,
   sender_user_id INT,
   receiver_user_id INT,
   currency_id INT,
   importe DECIMAL(10,2) NOT NULL,
   transaction_date DATETIME NOT NULL,

   FOREIGN KEY (sender_user_id) REFERENCES usuario(user_id),
   FOREIGN KEY (receiver_user_id) REFERENCES usuario(user_id),
   FOREIGN KEY (currency_id) REFERENCES moneda(currency_id)
);
```

Campos

Campo	Descripción
transaction_id	Identificador de la transacción
sender_user_id	Usuario que envía el dinero
receiver_user_id	Usuario que recibe el dinero
currency_id	Moneda utilizada
importe	Monto de la transacción
transaction_date	Fecha de la transacción

5. Inserción de Datos (DML)

Para probar el funcionamiento del sistema se insertaron registros de ejemplo en las tablas.

Insertar usuarios

INSERT INTO usuario (nombre, correo, password, saldo)

```
VALUES
('Juan Perez', 'juan@email.com', '1234', 5000),
('Maria Lopez', 'maria@email.com', 'abcd', 3000),
('Carlos Diaz', 'carlos@email.com', 'pass', 7000);
```

Insertar monedas

```
INSERT INTO moneda (currency_name, currency_symbol)
VALUES
('Peso Chileno', 'CLP'),
('Dolar', 'USD'),
('Euro', 'EUR');
```

Insertar transacciones

```
INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe,
transaction_date)
VALUES
(1,2,1,1000,NOW()),
(2,3,2,200,NOW()),
(3,1,1,500,NOW());
```

Estos datos permiten realizar pruebas de consultas y verificar el correcto funcionamiento de las relaciones entre tablas.

6. Consultas SQL

A continuación se presentan algunas consultas utilizadas para obtener información desde la base de datos.

Obtener el nombre de la moneda utilizada por un usuario

```
SELECT u.nombre, m.currency_name
FROM usuario u
JOIN transaccion t ON u.user_id = t.sender_user_id
JOIN moneda m ON t.currency_id = m.currency_id
```

```
WHERE u.user_id = 1;
```

Esta consulta permite identificar qué moneda utiliza un usuario en sus transacciones.

Obtener todas las transacciones

```
SELECT * FROM transaccion;
```

Esta consulta muestra todos los registros almacenados en la tabla de transacciones.

Obtener transacciones realizadas por un usuario específico

```
SELECT *
FROM transaccion
WHERE sender_user_id = 1;
```

Permite visualizar todas las transacciones realizadas por un usuario determinado.

7. Modificación de Datos

Para modificar información en la base de datos se utilizan sentencias **UPDATE**.

Modificar el correo de un usuario

```
UPDATE usuario
SET correo = 'nuevo_correo@email.com'
WHERE user_id = 1;
```

Este comando actualiza el correo electrónico de un usuario específico.

8. Eliminación de Datos

Para eliminar registros se utiliza la sentencia **DELETE**.

Eliminar una transacción

```
DELETE FROM transaccion
WHERE transaction_id = 2;
```

Este comando elimina completamente un registro de la tabla de transacciones.

9. Propiedades ACID en la Base de Datos

Las bases de datos relacionales utilizan el principio **ACID** para garantizar la integridad de los datos.

Atomicidad
Garantiza que una transacción se ejecute completamente o no se ejecute en absoluto.

Consistencia
Asegura que los datos siempre se mantengan válidos según las reglas definidas.

Aislamiento
Las transacciones se ejecutan de forma independiente sin interferir entre ellas.

Durabilidad
Una vez confirmada una transacción, los datos permanecen almacenados incluso si ocurre un fallo en el sistema.

Estas propiedades aseguran el correcto funcionamiento del sistema de la wallet.

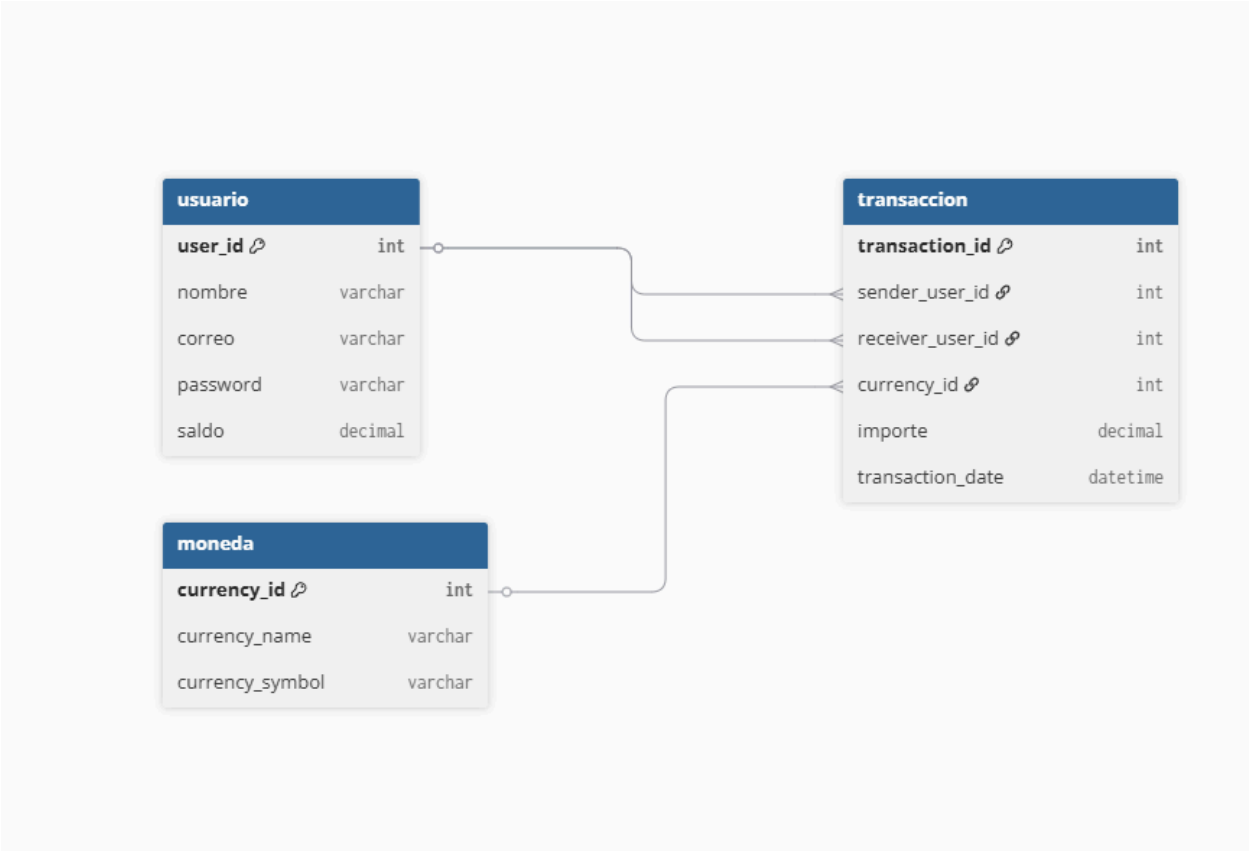
10. Conclusión

El desarrollo de este proyecto permitió aplicar conceptos fundamentales de bases de datos relacionales mediante el diseño e implementación de un sistema de almacenamiento para una billetera digital.

Se creó una base de datos con múltiples tablas relacionadas mediante claves primarias y foráneas, permitiendo mantener la integridad referencial entre los datos. Además, se realizaron operaciones de manipulación de datos mediante consultas SQL, incluyendo inserción, actualización, eliminación y recuperación de información.

Este proyecto demuestra cómo una base de datos relacional puede utilizarse para gestionar sistemas financieros de forma estructurada, segura y eficiente.

11. Evidencias




```
12:29:24    USE AlkeWallet
12:29:19    CREATE DATABASE AlkeWallet
```

```sql
DESCRIBE usuario;
DESCRIBE moneda;
DESCRIBE transaccion;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
transaction_id	int	NO	PRI	NULL	auto_increment
sender_user_id	int	YES	MUL	NULL	
receiver_user_id	int	YES	MUL	NULL	
currency_id	int	YES	MUL	NULL	
importe	decimal(10,2)	NO		NULL	
transaction_date	datetime	NO		NULL	

```
1 • DESCRIBE moneda;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
currency_id	int	NO	PRI	NULL	auto_increment
currency_name	varchar(50)	NO		NULL	
currency_symbol	varchar(10)	NO		NULL	

Limit to 1000 rows

```
1 • DESCRIBE usuario;
2
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
user_id	int	NO	PRI	NULL	auto_increment
nombre	varchar(100)	NO		NULL	
correo	varchar(100)	NO	UNI	NULL	
password	varchar(100)	NO		NULL	
saldo	decimal(10,2)	NO		NULL	

18	13:24:08	show tables	3 row(s) returned	0.000 sec / 0.000 sec
19	13:24:22	describe usuario	5 row(s) returned	0.000 sec / 0.000 sec
20	13:24:33	SELECT u.nombre, m.currency_name FROM usuario u JOIN transaccion t ON u.user_id = t.sender_user_id JOIN moneda m ON t.currency_id = m.curre...	1 row(s) returned	0.000 sec / 0.000 sec
21	13:25:01	SELECT * FROM transaccion LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
22	13:25:19	SELECT * FROM transaccion WHERE sender_user_id = 1 LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
23	13:25:40	UPDATE usuario SET correo = 'nuevo_correo@email.com' WHERE user_id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
24	13:25:59	DELETE FROM transaccion WHERE transaction_id = 2	0 row(s) affected	0.000 sec
25	13:26:18	START TRANSACTION	0 row(s) affected	0.000 sec
26	13:26:18	UPDATE usuario SET saldo = saldo - 500 WHERE user_id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
27	13:26:18	UPDATE usuario SET saldo = saldo + 500 WHERE user_id = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
28	13:26:18	COMMIT	0 row(s) affected	0.000 sec

12:29:24	USE AlkeWallet
12:29:19	CREATE DATABASE AlkeWallet

```
1 •  SHOW TABLES;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Tables_in_alkewallet
▶	moneda
	transaccion
	usuario